

Bonnes pratiques Git & GitHub

Guide personnel & travail en équipe — projet **Horus-Lab**

main est sacré · tout passe par une Pull Request

Dépôt : `horus-lab-team-s/website-horus-lab` · Équipe · CI/CD via GHCR

Objectif : ne plus jamais galérer à revenir à une version antérieure

1. Le principe d'or

La branche **main** est **SACRÉE**. Elle doit être **toujours déployable** et **toujours verte** (la CI passe). On n'y écrit **jamais** en direct : tout changement arrive par une **branche** puis une **Pull Request (PR)** relue par l'équipe.

Pourquoi c'est vital en équipe :

- **Relecture** : un coéquipier valide avant que ça entre dans `main`.
- **CI** : les tests / le build doivent réussir avant le merge.
- **Historique propre** : chaque fonctionnalité = une PR traçable.
- **Rollback facile** : annuler une PR est simple et sûr (voir §6).
- **Zéro écrasement** : personne ne casse le travail des autres.

2. Le workflow au quotidien

Le diagramme illustre le workflow quotidien. Une branche `main` est représentée par un point noir, avec une flèche horizontale à droite indiquant qu'elle est protégée et que personne ne pousse directement dessus. Une autre branche, `feat/xxx`, est représentée par un point noir et une flèche descendante vers la gauche. Une flèche ascendante pointe de `feat/xxx` vers `main`, accompagnée du texte `/ ^`. À droite de `feat/xxx`, une séquence de flèches horizontales mène à `Pull Request`, puis à `review + CI`, et enfin à `merge (sur GitHub)`.

Les 8 étapes, à répéter pour **chaque** fonctionnalité ou correctif :

```
# 1. Partir d'un main à jour
git checkout main
git pull origin main

# 2. Créer une branche dédiée
git checkout -b feat/section-actualites

# 3. Travailler, puis committer (petits commits clairs)
git add .
git commit -m "feat(home): ajoute la section actualites"

# 4. Pousser la BRANCHE (jamais main)
git push -u origin feat/section-actualites
```

5. Sur **github.com** → bandeau jaune « **Compare & pull request** » → ouvre la PR (titre clair + description : *quoi* et *pourquoi*) → assigne un relecteur.
6. L'équipe **relit**, la **CI tourne**. On discute/corrige dans la PR si besoin.
7. Quand c'est approuvé + CI verte → bouton « **Squash and merge** » sur GitHub → **main** est mis à jour.

```
# 8. Nettoyer en local
git checkout main
git pull origin main
git branch -d feat/section-actualites # supprime la branche locale
```

3. Conventions de nommage

Branches

Préfixe	Usage	Exemple
feat/	nouvelle fonctionnalité	feat/section-actualites
fix/	correction de bug	fix/hero-image-manquante
chore/	maintenance, config, deps	chore/maj-dependances
docs/	documentation	docs/guide-deploiement
hotfix/	correctif urgent en prod	hotfix/500-page-contact

Nom court, en minuscules, mots séparés par des tirets. Une branche = un sujet.

Messages de commit (Conventional Commits)

Format : **type(portée): description courte à l'impératif**

```
feat(blog): ajoute le forum public sous les articles
fix(auth): corrige la redirection apres connexion
docs(readme): explique le lancement en local
chore(ci): epingle la version de npm
refactor(cms): simplifie le fetch des services
```

Types courants : **feat**, **fix**, **docs**, **chore**, **refactor**, **test**, **style**.

4. Protéger **main** sur GitHub (le point pro)

C'est **LE** réglage qui empêche **tout le monde** (toi compris) de pousser en direct sur **main**. À faire une fois, par un admin du dépôt :

1. Dépôt sur GitHub → **Settings** → **Branches** → **Add branch ruleset** (ou « Add rule »).
2. Cible : la branche **main**.
3. Coche « **Require a pull request before merging** » → et « **Require approvals** » : au moins **1**.
4. Coche « **Require status checks to pass** » → sélectionne les checks CI (workflows **backend** / **frontend**).
5. Coche « **Block force pushes** » (interdit **git push --force** sur main).

6. (Recommandé) « **Do not allow bypassing the above settings** » → la règle s'applique **aussi aux admins**.

Résultat : un `git push origin main` direct sera **refusé**. La seule porte d'entrée vers `main` devient la Pull Request approuvée + CI verte. Fini les accidents.

5. La Pull Request — bonnes pratiques

- **Petite et ciblée** : une PR = un sujet. Plus c'est petit, plus c'est vite relu.
- **Titre + description clairs** : *quoi, pourquoi*, comment tester. Lier l'issue (`Closes #12`).
- **Assigner un relecteur** et attendre son **approbation**.
- **Répondre aux commentaires** et re-pousser sur la même branche (la PR se met à jour toute seule).
- « **Squash and merge** » : regroupe les commits de la PR en un seul propre sur `main`.
- **Supprimer la branche** après le merge (bouton sur GitHub).

6. Revenir en arrière proprement (rollback)

Le nerf de la guerre — sans jamais casser l'historique partagé.

Méthode	Quand	Comment
Revert d'une PR	annuler une PR déjà mergée	Sur GitHub, ouvre la PR mergée → bouton « Revert » → crée une PR d'annulation à merger.
<code>git revert</code>	annuler un commit précis	<code>git revert <sha></code> (crée un commit inverse, sûr) — sur une branche + PR.
Tags / Releases	marquer des versions stables	<code>git tag v1.0.0 && git push --tags</code> → repère clair pour revenir.
Épingler l'image (digest)	figer la version EN PROD	<code>BACKEND_IMAGE=...@sha256:...</code> dans le <code>.env</code> du serveur.

À NE JAMAIS FAIRE sur une branche partagée (surtout `main`) :

- `git push --force` (réécrit l'historique des autres)
- `git reset --hard` puis push (efface des commits publiés)
- committer un secret (`.env` , token `ghp_...`) — si ça arrive, **révoque-le tout de suite**.

7. Sécurité & hygiène

- **Jamais de secrets dans Git** : `.env` , tokens, mots de passe → couverts par `.gitignore` + **GitHub Secrets** pour la CI.
- **Un digest `sha256:` n'est pas un secret** (empreinte publique) ; un **PAT `ghp_...`** l'est.
- **Pull avant de commencer** chaque jour (`git pull`) pour rester synchro avec l'équipe.
- **Branches courtes** : merge vite, évite les branches qui divergent pendant des semaines.

8. Votre cas — Horus-Lab en équipe (mémo)

À faire une fois (admin) : activer la **protection de `main`** (§4) → plus personne ne pousse en direct.

Puis, à chaque tâche, le cycle complet :

```
git checkout main && git pull origin main
git checkout -b feat/mon-sujet
# ... travail + commits ...
git push -u origin feat/mon-sujet
# → sur GitHub : ouvrir la PR, faire relire, CI verte, "Squash and merge"
git checkout main && git pull origin main
git branch -d feat/mon-sujet
```

Bénéfice concret pour toi : revenir en arrière devient trivial (Revert d'une PR en 2 clics), l'équipe valide tout, `main` reste toujours sain, et la prod se pilote par images figées (digest). Plus jamais de « je n'arrive pas à revenir à ma version ».